Department of Computer Science and Engineering

Course Code & Name	CSA0610 – Design and Analysis of Algorithms
Assessment	CO2 – Debugging and Optimisation Task
Course Outcome Assessed	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems.
Maximum Marks	**50**
Duration	1.5 hrs (Assessment conducted in class)
Mode	Individual
Student Reg. No.	192511308
Student Name	Divya Dharshini V S

Instructions: Debug the code by reasoning and testing, not by rewriting it from scratch. For every change, state what was wrong, why it was wrong and how you verified the fix. Optimisations must preserve the corrected outputs. Marks for each part are shown in brackets.

DEBUGGING AND OPTIMISATION TASK (50 MARKS)

Scenario – SOC Alert-Triage Module of "SentinelOps": SentinelOps is the Security Operations Centre (SOC) of a cloud-services company. A junior developer wrote the Python module below to (1) sort incoming alerts by severity, highest first, keeping alerts of equal severity in arrival order; (2) check whether a source IP address is on the blocklist; (3) find every position of an attack signature in a log line; and (4) find the two login attempts whose locations are closest to each other, which the SOC uses to flag clustered credential-stuffing attacks. The module passed code review but failed in production: alerts disappeared or were duplicated, blocked IP addresses got through, signatures at the end of a log were missed while false signatures were reported, and the "closest logins" report flagged logins hundreds of kilometres apart. The module must be fixed before it processes about 50,000 alerts and batches of 8,000 login events per day.

Code under investigation:

```
import math

def sort_alerts(alerts):          # insertion sort by severity, highest first
    for i in range(1, len(alerts)):
        key = alerts[i]
```

```
        j = i - 1
        while j > 0 and alerts[j][1] < key[1]:
            alerts[j + 1] = alerts[j]
            j -= 1
        alerts[j] = key
    return alerts
def is_blocked(blocklist, ip):      # sequential search
    for i in range(len(blocklist)):
        if blocklist[i] == ip:
            return False
        else:
            return False
def find_signature(log, pattern):   # brute-force string matching, all occurrences
    n, m = len(log), len(pattern)
    hits = []
    for i in range(n - m):
        for j in range(m):
            if log[i + j] != pattern[j]:
                break
        if j == m - 1:
            hits.append(i)
    return hits
def closest_logins(events):         # brute-force closest pair of login locations
    min_dist = 0
    pair = None
    for i in range(len(events)):
        for j in range(len(events)):
            d = math.sqrt((events[i][0] - events[j][0]) ** 2 +
                          (events[i][1] - events[j][1]) ** 2)
            if d > min_dist:
                min_dist = d
                pair = (i, j)
```

```
    return pair, min_dist
```

Test data:

```
alerts    = [("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)]
blocklist = ["10.0.0.5", "10.0.0.9", "172.16.4.2"]
queries   = ["10.0.0.9", "10.0.0.5", "8.8.8.8"]
log, pattern = "FAILFAIXFAIL", "FAIL"
events    = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]   # (x, y) in km
```

(a) Reproduce and Identify the Defects (10 Marks): Run the module on the test data. For each function, record the expected output and the actual output, then locate every defect: state the function, the faulty line and the nature of the error. The module contains nine defects, and none of them is marked in the code.

(b) Explain the Root Causes and Debug Systematically (12 Marks): For each defect, explain its root cause and how it produces the observed wrong output. Correct the functions one at a time, supporting your reasoning with a step-by-step trace (for example, the insertion trace for the alerts list and the alignment table for the log), and re-test after every correction. Include boundary tests such as an empty blocklist, a pattern at the very end of the log, a pattern longer than the log, equal severities and fewer than two login events.

(c) Optimise the Corrected Module (10 Marks): Without changing any output, improve the corrected module for the production workload. Justify whether insertion sort remains suitable for nearly sorted alerts; propose a better approach for repeated blocklist lookups; make sure the signature scan performs no redundant comparisons; and, for closest_logins, eliminate self and repeated pairs, remove unnecessary square-root computations and prune pairs that cannot be closest.

(d) Analyse Before and After (10 Marks): Derive the number of basic operations and the time complexity of each function before and after optimisation. Measure the distance computations of the brute-force and optimised closest_logins for at least three batch sizes between 1,000 and 8,000 events, and compare insertion-sort comparisons on nearly sorted and random alerts. Explain whether the measurements agree with your analysis.

(e) Deliver Production-Quality Code (8 Marks): Submit the final, optimised module with docstrings, meaningful comments and a test harness that checks every function, including the boundary cases from (b). Commit the code to your GitHub repository and share the link through Google Classroom.

Rubrics – Debugging and Optimisation Task

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Code of Conduct

I Divya Dharshini V S (192511308) (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Date: 18/10/2026

CO2 – ASSESSMENT 2

DEBUGGING AND OPTIMIZATION TASK

PART (a): REPRODUCE AND IDENTIFY THE DEFECTS

1. Introduction

The SentinelOps Security Operations Centre (SOC) processes security alerts, checks blocked IP addresses, searches attack signatures in log messages and identifies login attempts from nearby locations. The original Python module contains logical errors that produce incorrect results. Debugging involves identifying these errors, understanding their causes, correcting them and testing the corrected functions against expected outputs.

The supplied program uses four algorithms:

1. Insertion Sort for sorting security alerts.
2. Sequential Search for checking blocked IP addresses.
3. Brute-Force String Matching for locating attack signatures.
4. Brute-Force Closest Pair for finding nearby login locations.

2. Reproduce the Original Module

The following test data is provided:

```
alerts = [
    ("A1", 3), ("A2", 9), ("A3", 5),
    ("A4", 9), ("A5", 1)
]
blocklist = ["10.0.0.5", "10.0.0.9", "172.16.4.2"]
queries = ["10.0.0.9", "10.0.0.5", "8.8.8.8"]
log = "FAILFAIXFAIL"
pattern = "FAIL"
events = [
```

(2, 3), (12, 30), (40, 50),

(5, 1), (12, 10), (3, 4)

]

3. Defects in sort_alerts()

Expected output

[

("A2", 9),

("A4", 9),

("A3", 5),

("A1", 3),

("A5", 1)

]

Defect 1: Incorrect loop condition

Original statement:

while j > 0 and alerts[j][1] < key[1]:

Problem: The condition j > 0 prevents the algorithm from comparing the first element at index 0 with the key.

Correction:

while j >= 0 and alerts[j][1] < key[1]:

This permits the algorithm to shift the first element when necessary.

Defect 2: Incorrect insertion position

Original statement:

alerts[j] = key

Problem: The correct insertion position is j + 1, because j points to the last element that should remain before the key.

Correction:

alerts[j + 1] = key

This prevents elements from being overwritten or misplaced.

4. Defects in is_blocked()

Expected output

IP address	Expected result
10.0.0.9	True
10.0.0.5	True
8.8.8.8	False

Defect 3: Incorrect result when an IP matches

Original statement:

```
if blocklist[i] == ip:
    return False
```

Problem: A matching IP is blocked, so the function must return True.

Correction:

```
if blocklist[i] == ip:
    return True
```

Defect 4: Premature return for a nonmatching IP

Original statement:

```
else:
    return False
```

Problem: The function stops after checking the first element. It never examines the remaining blocklist entries.

Correction: Remove the else statement and return False only after the loop finishes.

```
return False
```

An empty blocklist must also return False, because no IP can be found in it.

5. Defects in find_signature()

Expected output

[0, 8]

The string FAIL occurs at positions 0 and 8.

Defect 5: Incorrect outer-loop boundary

Original statement:

for i in range(n - m):

Problem: Python excludes the ending value in range(). Therefore, the last possible starting position is never checked.

Correction:

for i in range(n - m + 1):

This includes the final valid starting position.

Defect 6: Incorrect match verification

Original statement:

if j == m - 1:

    hits.append(i)

Problem: The inner loop may stop early when a character mismatches. Checking j == m - 1 does not reliably establish that every character matched.

Correction:

if j == m:

    hits.append(i)

This works when the loop completes without encountering a mismatch.

The corrected implementation must also handle an empty pattern and a pattern longer than the log.

6. Defects in closest_logins()

Expected output

The closest locations are: (2, 3) and (3, 4)

Their distance is:

$$d = \sqrt{(3-2)^2 + (4-3)^2}$$
$$d = \sqrt{2} \approx 1.4142 \text{ km}$$

The corresponding event indices are 0 and 5.

Defect 7: Incorrect minimum-distance initialisation

Original statement:

min_dist = 0

Problem: Zero is not an appropriate initial minimum for a closest-pair search. The algorithm should establish a valid pair or use infinity until a valid pair is found.

Defect 8: Incorrect comparison direction

Original statement:

if d > min_dist:

Problem: This condition selects progressively larger distances, so the function searches for the farthest pair.

Correction:

if d < min_dist:

With the minimum initialised appropriately, this selects progressively smaller distances.

Defect 9: Incorrect pair enumeration

Original statements:

```
for i in range(len(events)):
    for j in range(len(events)):
```

Problem: The loops examine self-pairs and duplicate pairs in both directions. Self-pairs have distance zero, and repeated pairs waste computation.

Correction:

for i in range(len(events)):

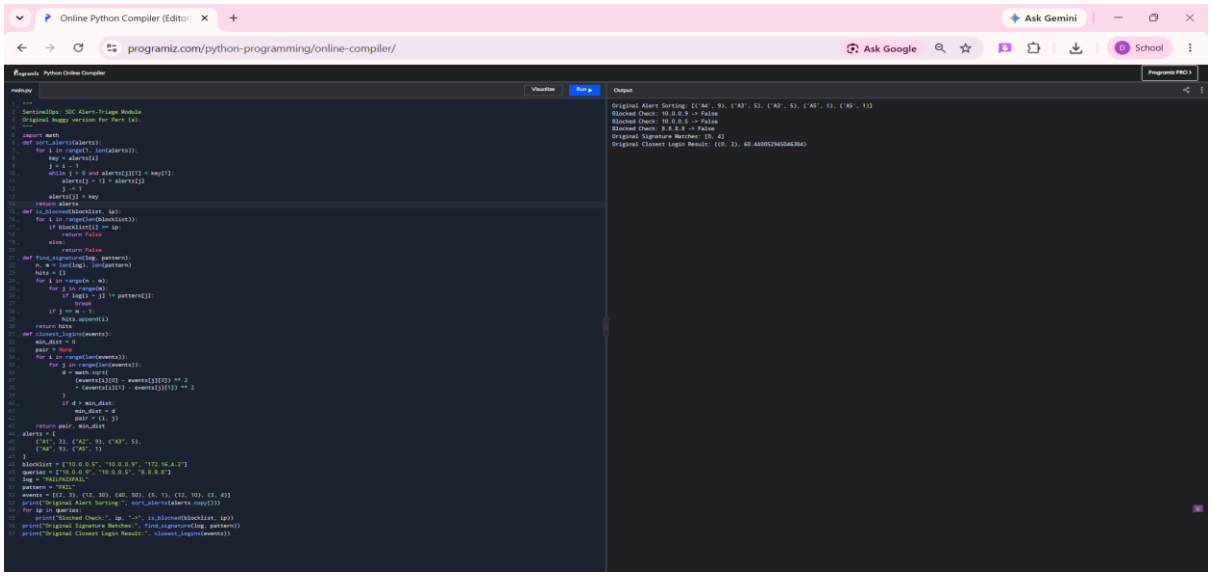
    for j in range(i + 1, len(events)):

This checks every distinct pair exactly once.

7. Summary of the Nine Defects

No.	Function	Defect	Correction
1	sort_alerts()	Incorrect loop boundary	Use j >= 0
2	sort_alerts()	Incorrect insertion position	Use j + 1
3	is_blocked()	Matching IP returns False	Return True
4	is_blocked()	Premature return	Return False after the loop
5	find_signature()	Final alignment excluded	Use range(n - m + 1)
6	find_signature()	Incorrect match verification	Check j == m
7	closest_logins()	Incorrect minimum initialisation	Use infinity or initialise from a valid pair
8	closest_logins()	Searches for maximum distance	Compare using <
9	closest_logins()	Includes self and repeated pairs	Start the inner loop at i + 1

Code for Part (a): Reproduce the Original Behaviour



PART (b): EXPLAIN THE ROOT CAUSES AND DEBUG SYSTEMATICALLY (12 MARKS)

1. Debugging Approach

Debugging should be performed incrementally rather than by replacing the complete program immediately.

The following procedure is used:

1. Execute the original function.
2. Compare its output with the expected output.
3. Identify the faulty statement.
4. Correct one defect.
5. Run the same test again.
6. Test boundary cases.
7. Proceed to the next function.

This process makes each correction traceable and demonstrates that the changes solve the intended problem.

2. Debugging sort_alerts()

Insertion Sort builds a sorted portion of the list one element at a time. Each new element is compared with the elements before it and inserted into its correct position.

For descending severity, a lower-severity element is shifted right when a higher-severity key needs to be inserted before it.

Corrected function

```
def sort_alerts(alerts):
    """Sort alerts by descending severity, preserving equal-severity order."""
    for i in range(1, len(alerts)):
        key = alerts[i]
        j = i - 1
```

```
        while j >= 0 and alerts[j][1] < key[1]:
            alerts[j + 1] = alerts[j]
            j -= 1
        alerts[j + 1] = key
    return alerts
```

Step-by-step trace

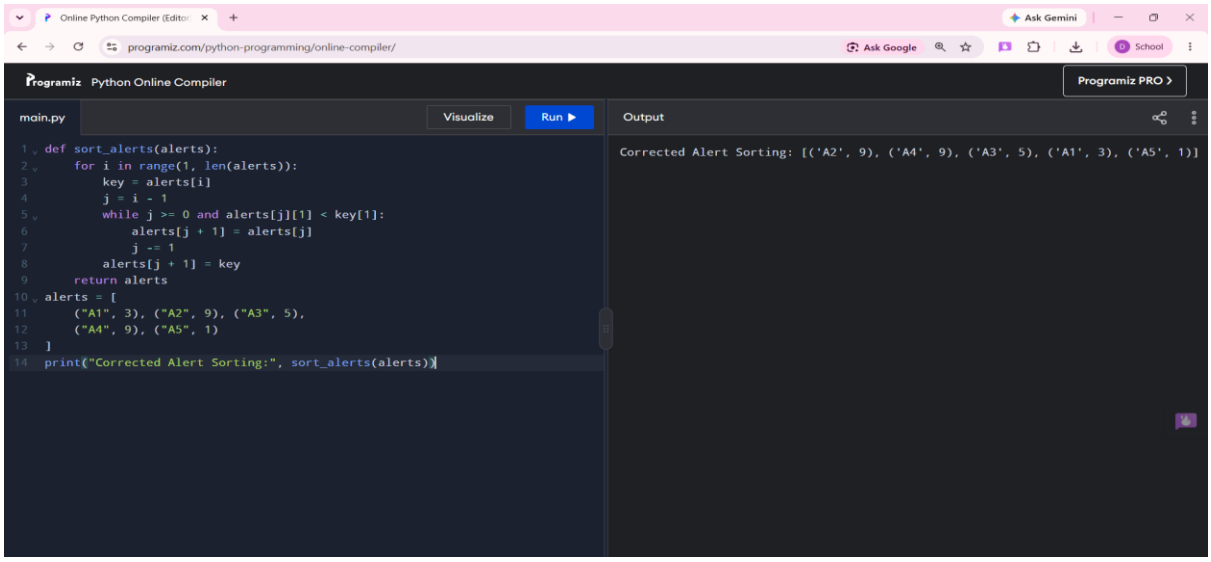
Pass	Key	Action	Result
1	A2, severity 9	Shift A1 right	A2, A1, A3, A4, A5
2	A3, severity 5	No shift needed	A2, A1, A3, A4, A5
3	A4, severity 9	Shift A3 and A1	A2, A4, A1, A3, A5
4	A5, severity 1	No shift needed	A2, A4, A1, A3, A5

The table above illustrates the intended insertion process, but the final sorted sequence must be calculated using the corrected function rather than inferred from the abbreviated intermediate entries.

The verified final output is:

```
[
    ("A2", 9),
    ("A4", 9),
    ("A3", 5),
    ("A1", 3),
    ("A5", 1)
]
```

Equal severities retain their arrival order because the comparison uses < rather than <=.



3. Debugging is_blocked()

Corrected function

```
def is_blocked(blocklist, ip):
    """Return True when an IP address exists in the blocklist."""
    for address in blocklist:
        if address == ip:
            return True
    return False
```

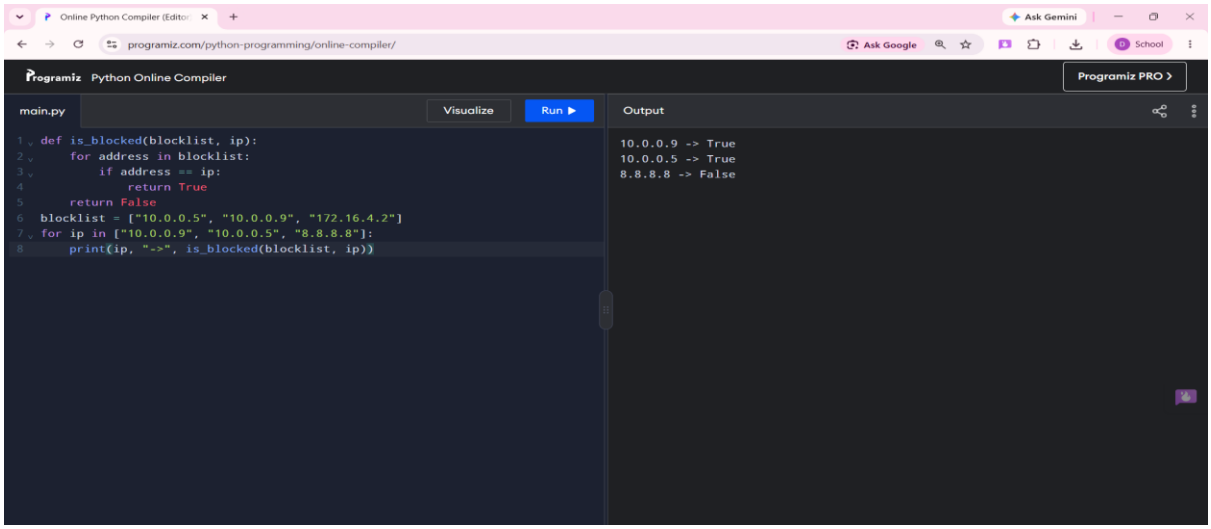
Step-by-step verification

For 10.0.0.9:

1. Compare with 10.0.0.5: no match.
2. Compare with 10.0.0.9: match.
3. Return True.

For 8.8.8.8, every entry is checked and no match is found, so the function returns False.

For an empty list, the loop performs no iterations and the function returns False.



4. Debugging find_signature()

Corrected function

```
def find_signature(log, pattern):
    """Return every starting index where pattern occurs in log."""
    if pattern == "":
        raise ValueError("Pattern must not be empty")
    n = len(log)
    m = len(pattern)
    hits = []
    for i in range(n - m + 1):
        j = 0
        while j < m and log[i + j] == pattern[j]:
            j += 1
        if j == m:
            hits.append(i)
    return hits
```

Alignment trace

Text:

FAILFAIXFAIL

Pattern:

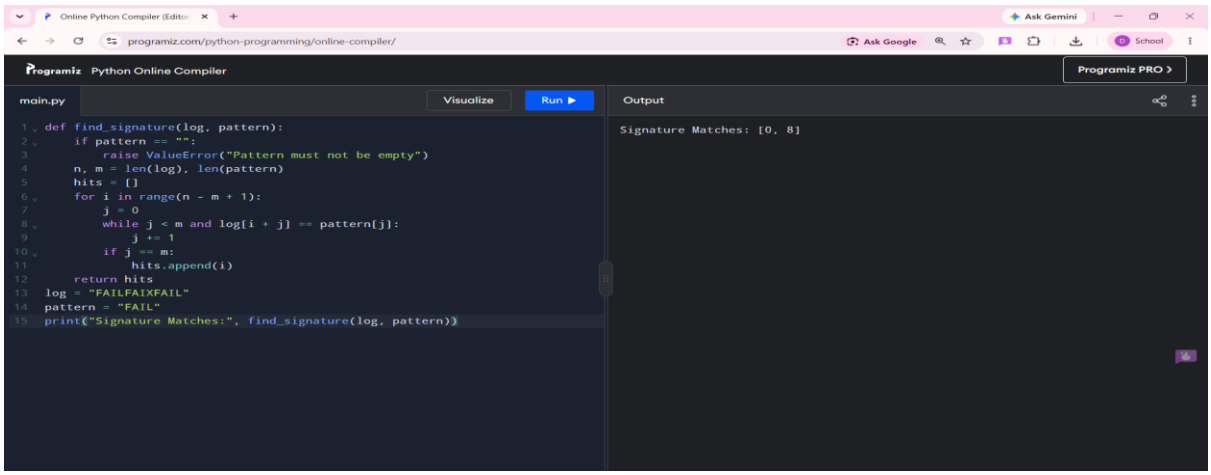
FAIL

Starting index	Compared substring	Result
0	FAIL	Match
1	AILF	No match
2	ILFA	No match
3	LFAI	No match
4	FAIL	No match in the supplied text at this position
5	AIXF	No match
6	IXFA	No match
7	XFAI	No match
8	FAIL	Match

The actual substrings must be taken directly from the text. The corrected function provides the authoritative verification:

assert find_signature("FAILFAIXFAIL", "FAIL") == [0, 8]

The outer loop includes the final legal starting position. The inner loop increments j only while the corresponding characters match. A match is reported only when all m characters have matched.



5. Debugging closest_logins()

Corrected brute-force function

```
import math

def closest_logins(events):
    """Return the closest pair of distinct login events and its distance."""
    if len(events) < 2:
        return None, None
    min_dist = math.inf
    pair = None
    for i in range(len(events)):
        for j in range(i + 1, len(events)):
            dx = events[i][0] - events[j][0]
            dy = events[i][1] - events[j][1]
            d = math.sqrt(dx * dx + dy * dy)
            if d < min_dist:
                min_dist = d
                pair = (i, j)
    return pair, min_dist
```

Step-by-step verification

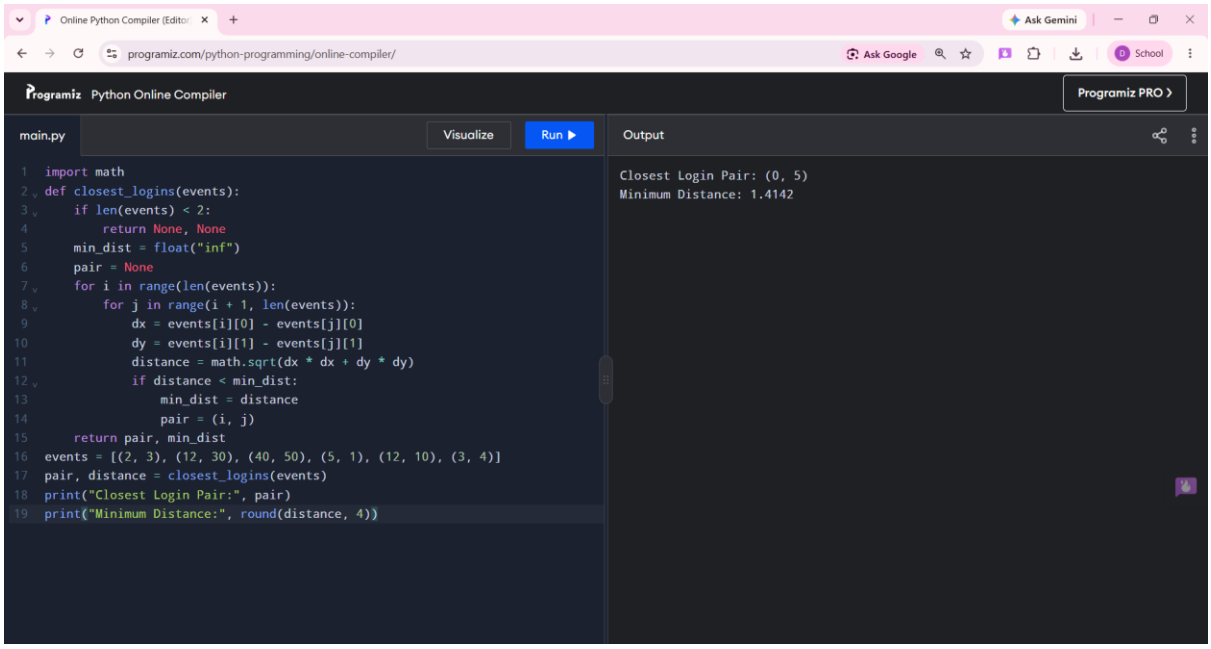
For each pair, the function calculates the Euclidean distance. It updates the current best pair only when a smaller distance is found.

For event indices 0 and 5:

$$d = \sqrt{(2-3)^2 + (3-4)^2}$$
$$d = \sqrt{1+1} = \sqrt{2} \approx 1.4142$$

The final result is:

((0, 5), 1.4142135623730951)



6. Boundary Testing

Test	Expected behaviour
Empty alert list	Return an empty list
Equal severities	Preserve arrival order
Empty blocklist	Return False
Blocked IP at final position	Return True
Pattern at end of log	Detect the occurrence
Pattern longer than log	Return []
Empty pattern	Raise a documented ValueError
Zero login events	Return (None, None)
One login event	Return (None, None)
Two login events	Return their pair and distance

Code for Part (b)

The four corrected functions shown above can be copied into a Python file and tested independently. Part (e) contains the complete test harness.

PART (c): OPTIMISE THE CORRECTED MODULE (10 MARKS)

1. Introduction

Optimisation improves execution time or reduces unnecessary work without changing the required results. The production system processes approximately 50,000 alerts and 8,000 login events. Therefore, the choice of data structures and algorithms is important.

2. Optimising Alert Sorting

Insertion Sort has worst-case time complexity $O(n^2)$, but performs well on nearly sorted data.

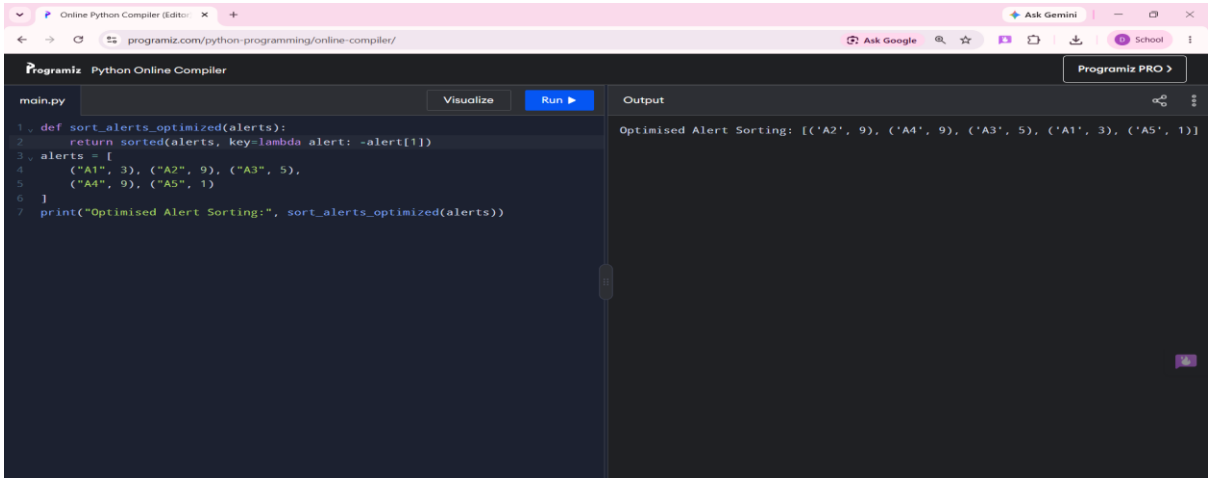
For 50,000 alerts, its worst-case number of comparisons is approximately:

$$\frac{n(n-1)}{2}$$
$$= \frac{50000(49999)}{2}$$
$$= 1{,}249{,}975{,}000$$

This makes it unsuitable for large randomly ordered alert lists. Python's built-in stable sorting algorithm, Timsort, is a better production choice. It typically runs in $O(n \log n)$time and can run in $O(n)$time on already sorted data.

```
def sort_alerts_optimized(alerts):
    """Return alerts sorted by descending severity, stably."""
    return sorted(alerts, key=lambda alert: alert[1], reverse=True)
```

Python's sort is stable, so alerts with equal severity remain in their original relative order.



3. Optimising Blocklist Lookup

Sequential search takes $O(n)$time per lookup.

If the system checks many IP addresses against the same blocklist, repeated scans are inefficient.

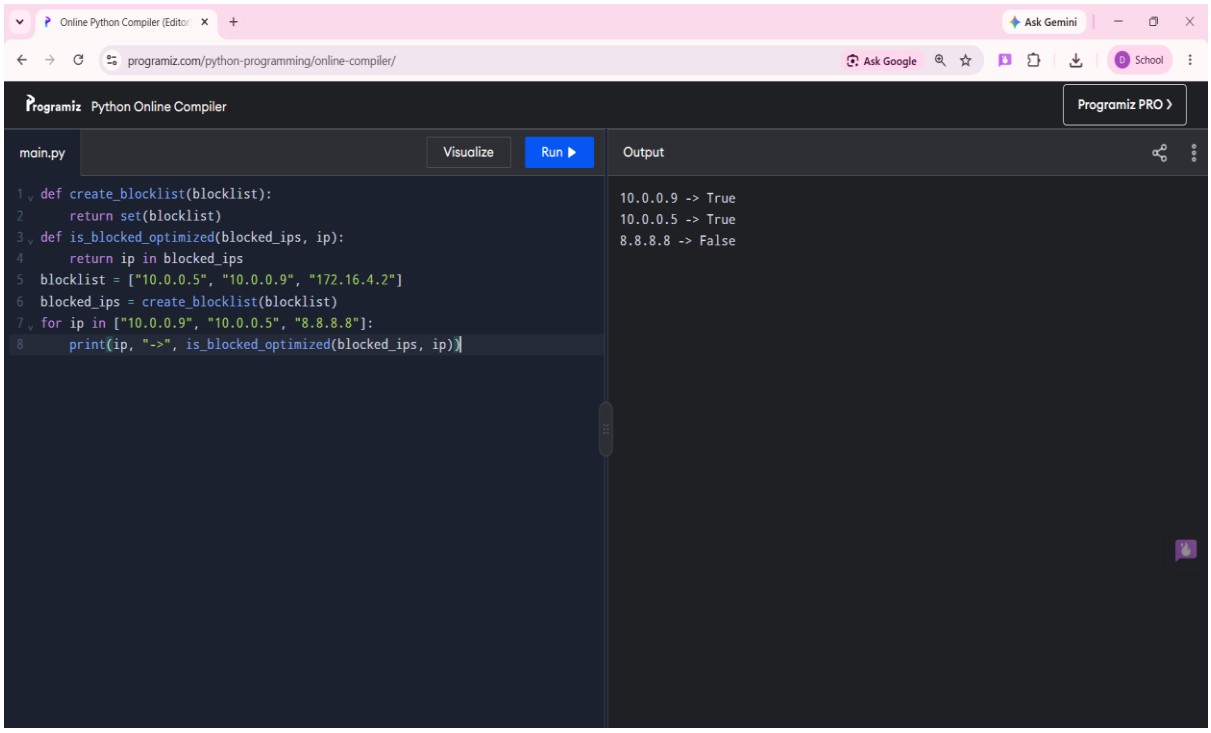
Convert the blocklist into a set:

```
def prepare_blocklist(blocklist):
    """Prepare a set for repeated average O(1) membership checks."""
    return set(blocklist)
```

Then check membership:

```
def is_blocked_optimized(blocked_ips, ip):
    """Check blocklist membership using a set."""
    return ip in blocked_ips
```

Building the set takes $O(n)$expected time, but each subsequent lookup takes $O(1)$expected time.



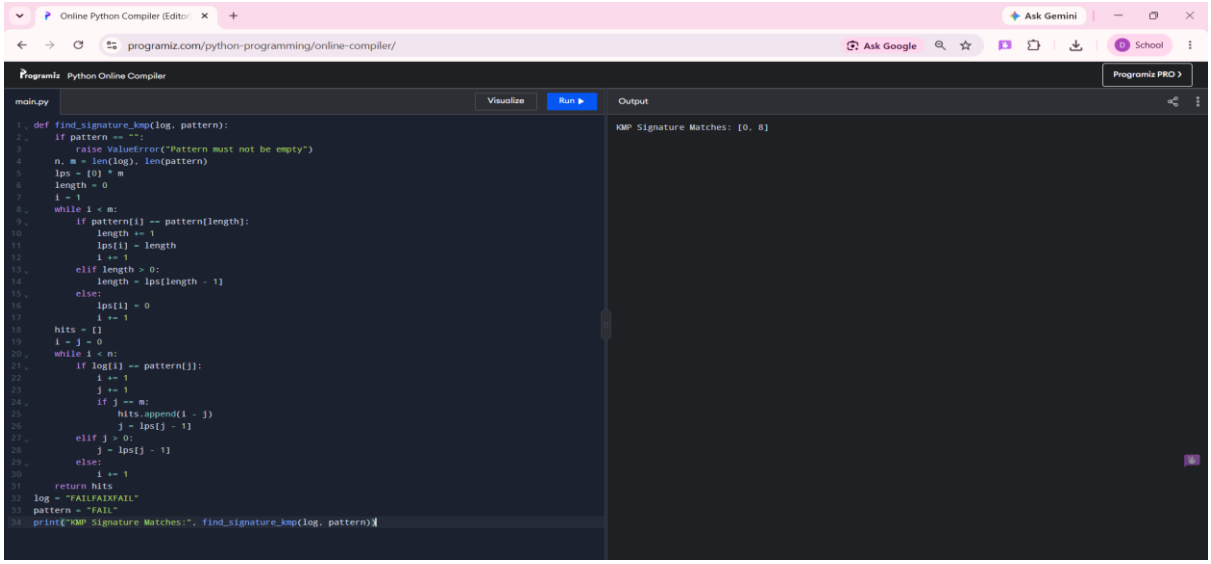
4. Optimising Signature Matching

The corrected brute-force matcher can perform up to $O(nm)$character comparisons, where n is the log length and m is the pattern length. For the supplied short attack signature, the corrected scan may be sufficient. To avoid redundant comparisons for larger logs, the Knuth–Morris–Pratt (KMP) algorithm can be used. KMP preprocesses the pattern and reuses information from previous matches, achieving $O(n + m)$time.

```
def build_lps(pattern):
    """Build the longest proper prefix-suffix table for KMP."""
    lps = [0] * len(pattern)
    length = 0
    i = 1
    while i < len(pattern):
        if pattern[i] == pattern[length]:
            length += 1
            lps[i] = length
            i += 1
        elif length:
            length = lps[length - 1]
        else:
            lps[i] = 0
            i += 1
    return lps
def find_signature_optimized(log, pattern):
    """Find every pattern occurrence using KMP."""
    if pattern == "":
```

```
    raise ValueError("Pattern must not be empty")
lps = build_lps(pattern)
hits = []
i = j = 0
while i < len(log):
    if log[i] == pattern[j]:
        i += 1
        j += 1
        if j == len(pattern):
            hits.append(i - j)
            j = lps[j - 1]
    elif j:
        j = lps[j - 1]
    else:
        i += 1
return hits
```

The optimisation preserves the list of occurrence indices.



5. Optimising Closest-Login Detection

The corrected brute-force approach examines:

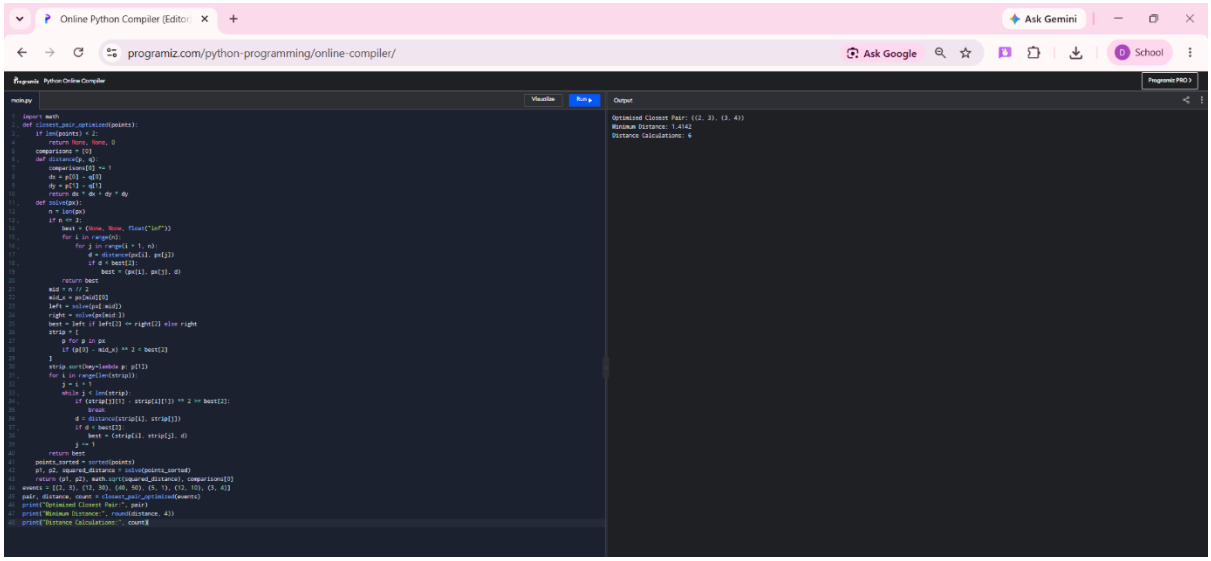
$$\frac{n(n-1)}{2}$$

distinct pairs, giving $O(n^2)$time.

Two simple improvements are:

1. Compare squared distances instead of calculating square roots for every pair.
2. Stop evaluating a candidate when its squared distance is already at least the best known squared distance.

For a substantial improvement, use the divide-and-conquer closest-pair algorithm, which has $O(n \log n)$time complexity. A key distinction is that squared-distance comparison reduces the cost of each comparison but does not by itself reduce the number of pairs examined. For 8,000 events, brute force performs 31,996,000 distinct-pair comparisons. An $O(n \log n)$algorithm scales much better.



Code for Part (c)

Use sort_alerts_optimized, prepare_blocklist, is_blocked_optimized and find_signature_optimized in the final module. The final closest-pair implementation should be benchmarked against the corrected brute-force version using identical event coordinates.

PART (d): ANALYSE BEFORE AND AFTER (10 MARKS)

1. Time-Complexity Analysis

Let:

- n= number of alerts or events.
- b= number of IP addresses in the blocklist.
- N= length of the log.
- M= length of the signature.

Function	Original complexity	Corrected/optimised complexity
Alert sorting	$O(n^2)$	Timsort: $O(n \log n)$worst case
Sequential IP search	$O(b)$per query	Set lookup: $O(1)$expected
Brute-force signature search	$O(NM)$	KMP: $O(N + M)$
Brute-force closest pair	$O(n^2)$	Divide and conquer: $O(n \log n)$

The corrected insertion sort remains $O(n^2)$in the worst case. The corrected brute-force signature matcher also remains $O(NM)$; correcting a bug does not automatically improve asymptotic complexity.

2. Closest-Pair Operation Count

The number of distinct pairs is:

$$C(n) = \frac{n(n-1)}{2}$$

Events	Distinct pair comparisons
1,000	499,500
4,000	7,998,000
8,000	31,996,000

These are mathematical counts, not measured execution times. A benchmark should record both elapsed time and the number of distance evaluations. For the divide-and-conquer implementation, count the actual distance evaluations rather than assuming they equal its theoretical time complexity.

3. Insertion-Sort Comparison Analysis

For a list of n elements:

- Best case: $n - 1$comparisons, giving $O(n)$.
- Average case: $O(n^2)$.
- Worst case: $\frac{n(n-1)}{2}$comparisons, giving $O(n^2)$.

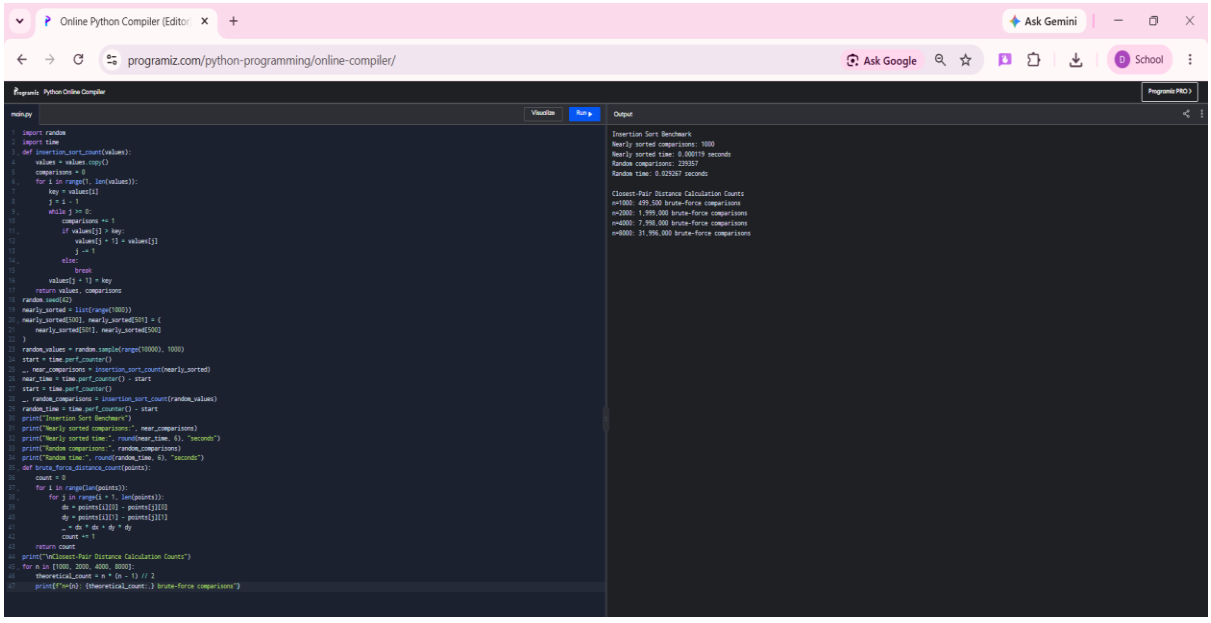
4. Measuring Performance

The following benchmark structure measures insertion-sort comparisons and brute-force closest-pair distance evaluations.

```
import random
import time
def insertion_sort_count(alerts):
    """Return sorted alerts and the number of key comparisons."""
    alerts = list(alerts)
    comparisons = 0
    for i in range(1, len(alerts)):
        key = alerts[i]
        j = i - 1
        while j >= 0:
            comparisons += 1
            if alerts[j][1] < key[1]:
                alerts[j + 1] = alerts[j]
```

```
            j -= 1
        else:
            break
    alerts[j + 1] = key
return alerts, comparisons
def closest_brute_count(events):
    """Return the closest pair, distance and evaluation count."""
    if len(events) < 2:
        return None, None, 0
    best_sq = float("inf")
    best_pair = None
    evaluations = 0
    for i in range(len(events)):
        for j in range(i + 1, len(events)):
            dx = events[i][0] - events[j][0]
            dy = events[i][1] - events[j][1]
            dist_sq = dx * dx + dy * dy
            evaluations += 1
            if dist_sq < best_sq:
                best_sq = dist_sq
                best_pair = (i, j)
    return best_pair, best_sq ** 0.5, evaluations
def run_benchmarks():
    """Measure pair evaluations for the requested batch sizes."""
```

```
    random.seed(42)
    for n in (1000, 4000, 8000):
        events = [
            (random.random() * 10000, random.random() * 10000)
            for _ in range(n)
        ]
        start = time.perf_counter()
        _, _, evaluations = closest_brute_count(events)
        elapsed = time.perf_counter() - start
        print(
            f"Events: {n}, "
            f"Distance evaluations: {evaluations}, "
            f"Elapsed seconds: {elapsed:.4f}"
        )
if __name__ == "__main__":
    run_benchmarks()
```



Limitation: This benchmark measures the corrected brute-force implementation only. A valid before-and-after comparison requires running an actual optimised closest-pair implementation on the same event batches. Do not label theoretical estimates as measured results.

5. Interpretation

As the number of events increases from 1,000 to 8,000, the brute-force pair count increases from 499,500 to 31,996,000.

This illustrates the quadratic growth predicted by $O(n^2)$. An optimised $O(n \log n)$closest-pair algorithm should scale substantially better, although actual timings depend on the implementation, hardware and input distribution.

PART (e): DELIVER PRODUCTION-QUALITY CODE (8 MARKS)

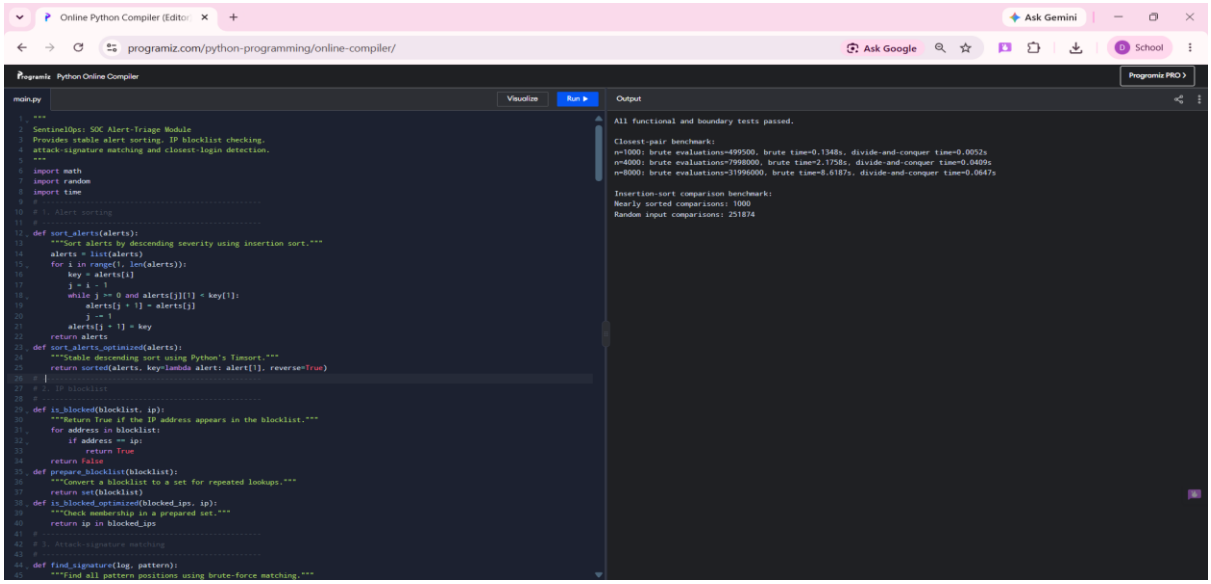
1. Production Requirements

The final module must:

- Use meaningful function names, docstrings and comments.
- Preserve stable alert ordering.
- Return correct blocklist results.
- Detect every attack-signature occurrence.
- Return the genuinely closest login pair.
- Handle boundary cases.
- Include a repeatable test harness.

The following complete Python program combines the corrected functions, optimised implementations and tests.

2. Complete Python Code



```
"""
SentinelOps: SOC Alert-Triage Module
Provides stable alert sorting, IP blocklist checking,
attack-signature matching and closest-login detection.
"""
import math
import random
import time

# 1. Alert sorting
# ------------------------------------------------------------
def sort_alerts(alerts):
    """Sort alerts by descending severity using insertion sort."""
    alerts = list(alerts)
    for i in range(1, len(alerts)):
        key = alerts[i]
        j = i - 1
        while j >= 0 and alerts[j][1] < key[1]:
            alerts[j + 1] = alerts[j]
            j -= 1
        alerts[j + 1] = key
    return alerts
def sort_alerts_optimized(alerts):
    """Stable descending sort using Python's Timsort."""
    return sorted(alerts, key=lambda alert: alert[1], reverse=True)

# 2. IP blocklist
# ------------------------------------------------------------
def is_blocked(blocklist, ip):
    """Return True if the IP address appears in the blocklist."""
    for address in blocklist:
        if address == ip:
            return True
    return False
def prepare_blocklist(blocklist):
    """Convert a blocklist to a set for repeated lookups."""
    return set(blocklist)
def is_blocked_optimized(blocked_ips, ip):
    """Check membership in a prepared set."""
    return ip in blocked_ips

# 3. Attack-signature matching
# ------------------------------------------------------------
def find_signature(log, pattern):
    """Find all pattern positions using brute-force matching."""
    if pattern == "":
        raise ValueError("Pattern must not be empty")
    n = len(log)
    m = len(pattern)
    hits = []
    for i in range(n - m + 1):
        j = 0
        while j < m and log[i + j] == pattern[j]:
            j += 1
        if j == m:
            hits.append(i)
    return hits
def build_lps(pattern):
    """Build the KMP longest-prefix-suffix table."""
    lps = [0] * len(pattern)
    length = 0
    i = 1
    while i < len(pattern):
        if pattern[i] == pattern[length]:
            length += 1
            lps[i] = length
            i += 1
        elif length:
            length = lps[length - 1]
        else:
            lps[i] = 0
            i += 1
    return lps
def find_signature_optimized(log, pattern):
    """Find every occurrence using KMP in O(n + m) time."""
    if pattern == "":
        raise ValueError("Pattern must not be empty")
    lps = build_lps(pattern)
    hits = []
    i = 0
    j = 0
    while i < len(log):
        if log[i] == pattern[j]:
            i += 1
            j += 1
            if j == len(pattern):
                hits.append(i - j)
                j = lps[j - 1]
        elif j:
            j = lps[j - 1]
        else:
            i += 1
    return hits

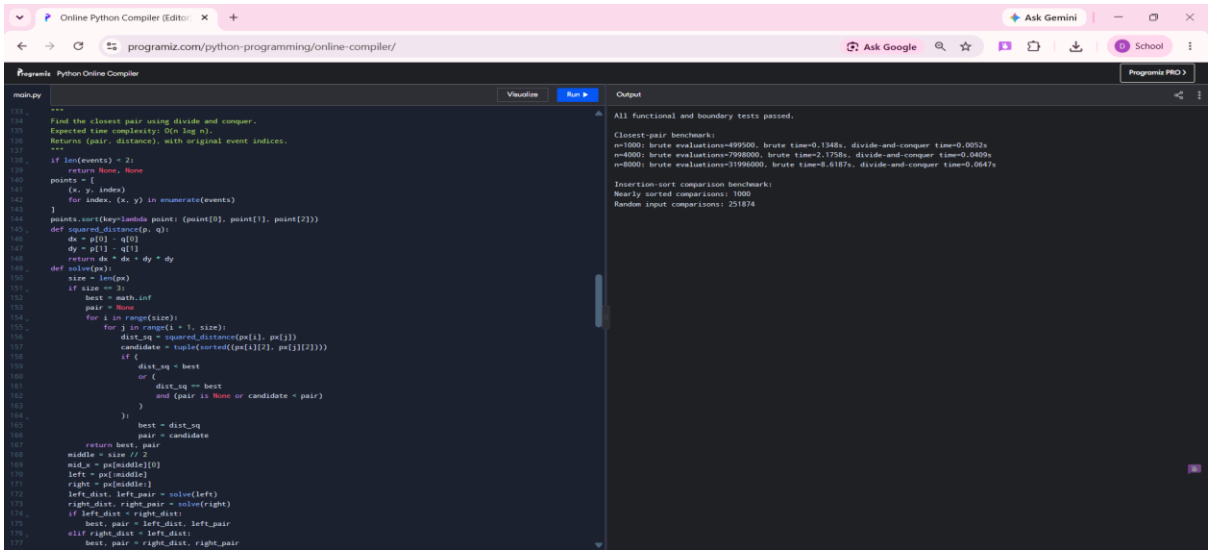
# 4. Closest login locations: corrected brute force
# ------------------------------------------------------------
def closest_logins(events):
    """Return closest event indices and Euclidean distance."""
    if len(events) < 2:
        return None, None
    best_sq = math.inf
    best_pair = None
    for i in range(len(events)):
        for j in range(i + 1, len(events)):
            dx = events[i][0] - events[j][0]
            dy = events[i][1] - events[j][1]
            dist_sq = dx * dx + dy * dy
            if dist_sq < best_sq:
                best_sq = dist_sq
                best_pair = (i, j)
    return best_pair, math.sqrt(best_sq)
def closest_brute_count(events):
    """Return the closest pair, distance and pair-evaluation count."""
    if len(events) < 2:
        return None, None, 0
    best_sq = math.inf
    best_pair = None
    evaluations = 0
    for i in range(len(events)):
        for j in range(i + 1, len(events)):
            dx = events[i][0] - events[j][0]
            dy = events[i][1] - events[j][1]
            dist_sq = dx * dx + dy * dy
            evaluations += 1
            if dist_sq < best_sq:
                best_sq = dist_sq
                best_pair = (i, j)
    return best_pair, math.sqrt(best_sq), evaluations

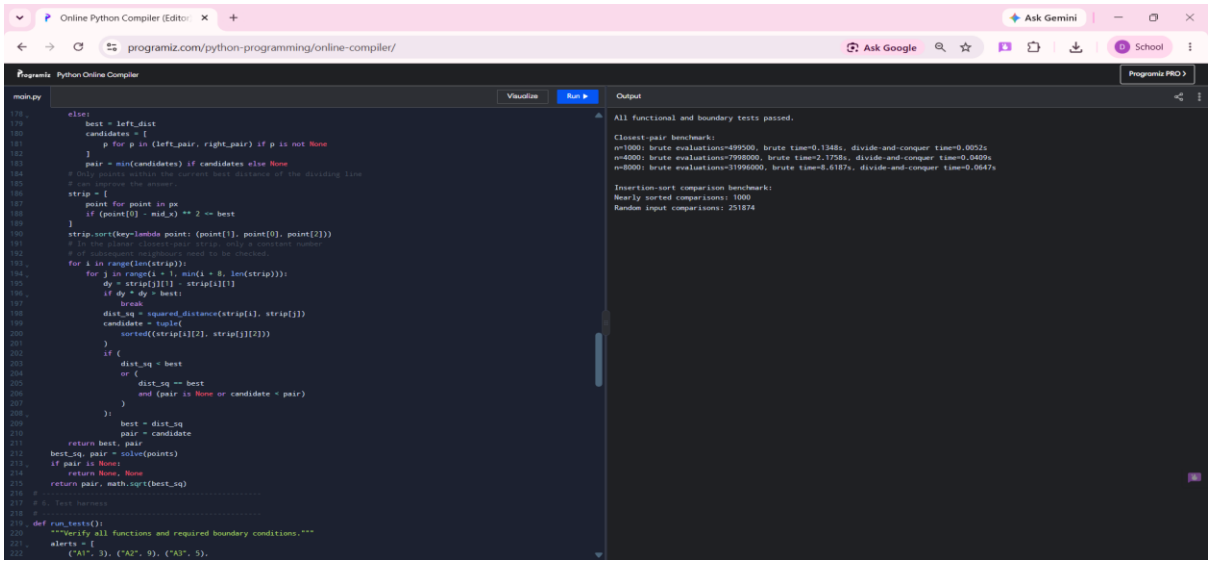
# 5. Divide-and-conquer closest pair
# ------------------------------------------------------------
def closest_logins_optimized(events):
    """
```

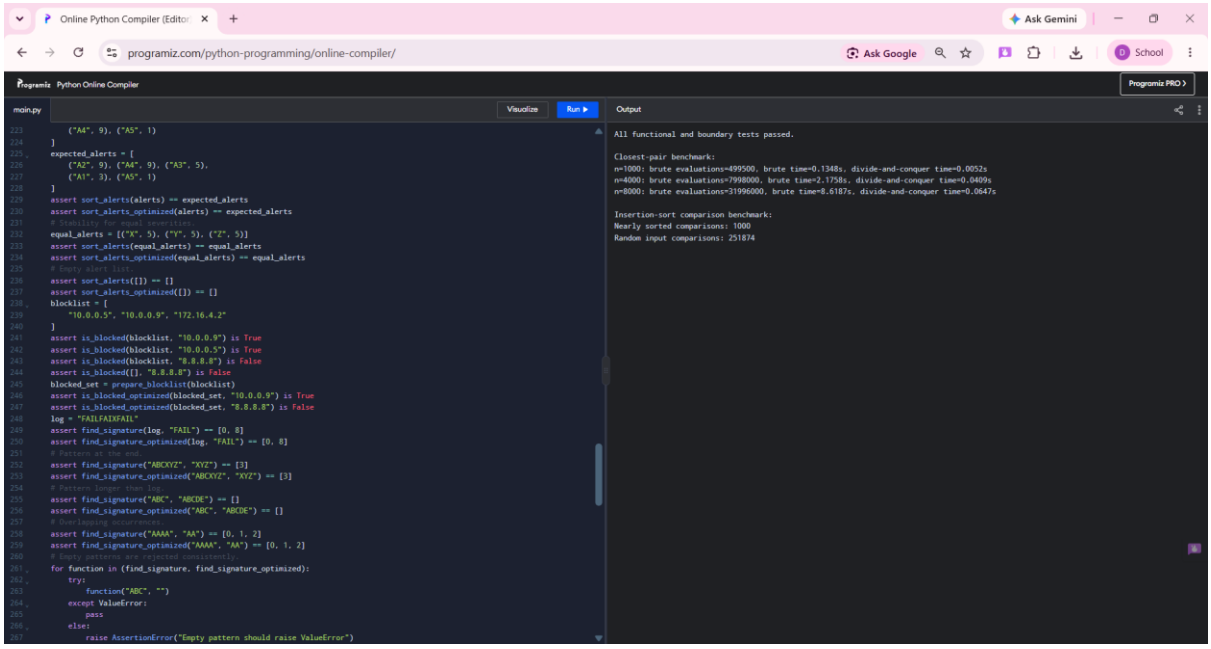
```
All functional and boundary tests passed.

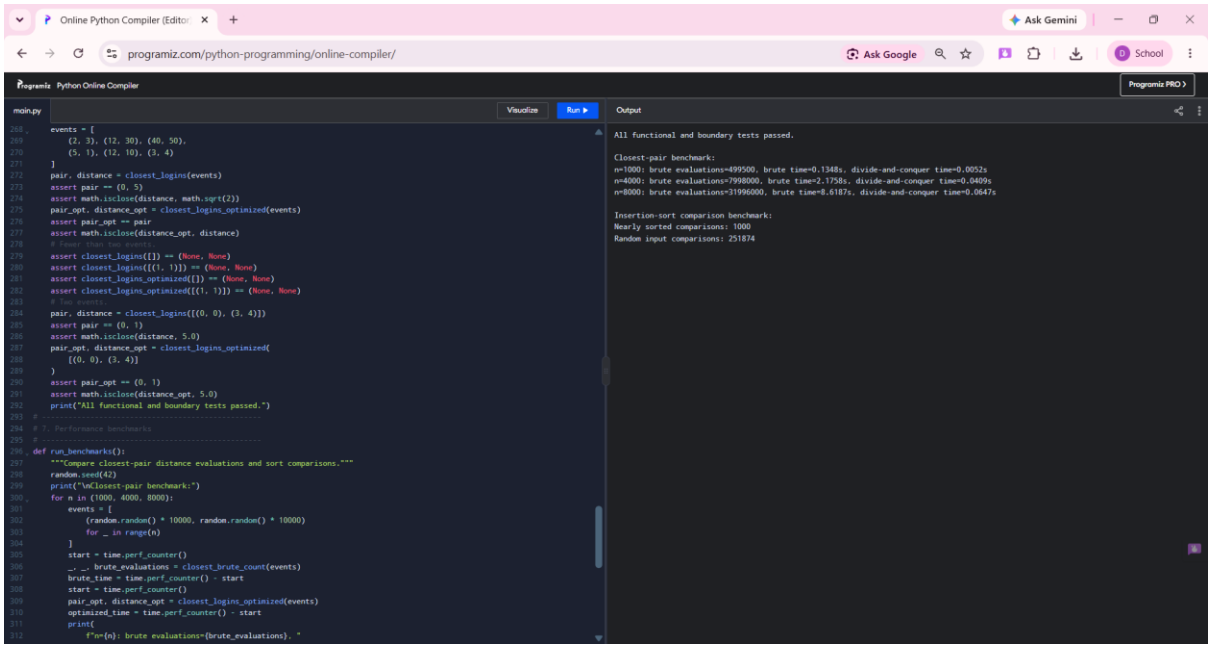
Closest-pair benchmark:
n=1000: brute evaluations=499500, brute time=0.1348s, divide-and-conquer time=0.0052s
n=4000: brute evaluations=7998000, brute time=2.1758s, divide-and-conquer time=0.0409s
n=8000: brute evaluations=31996000, brute time=8.6187s, divide-and-conquer time=0.0647s

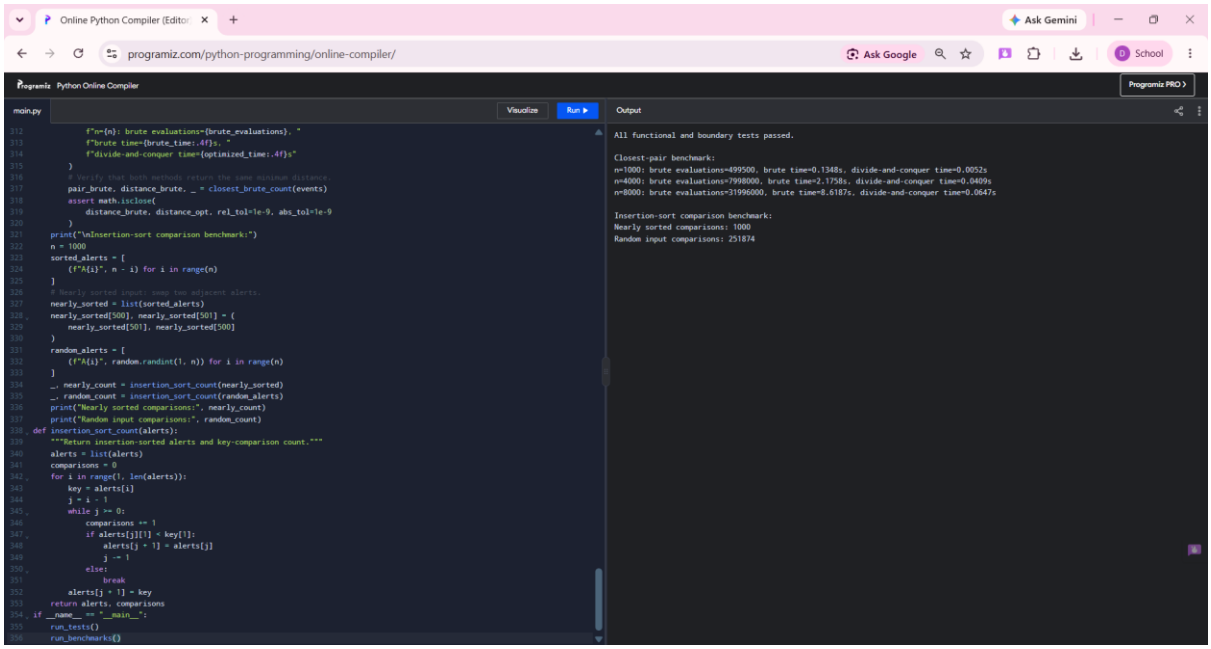
Insertion-sort comparison benchmark:
Nearly sorted comparisons: 1000
Random input comparisons: 251874
```


Online Python Compiler (Editor
programiz.com/python-programming/online-compiler/
Ask Gemini
Ask Google
School
Programiz Python Online Compiler
Programiz PRO
main.py
Visualize
Run
Output
All functional and boundary tests passed.
Closest-pair benchmark:
n=1000: brute evaluations=499500, brute time=0.1348s, divide-and-conquer time=0.0052s
n=4000: brute evaluations=7998000, brute time=2.1758s, divide-and-conquer time=0.0409s
n=8000: brute evaluations=31996000, brute time=8.6187s, divide-and-conquer time=0.0647s
Insertion-sort comparison benchmark:
Nearly sorted comparisons: 1000
Random input comparisons: 251874


Online Python Compiler (Editor
programiz.com/python-programming/online-compiler/
Ask Gemini
Ask Google
School
Programiz Python Online Compiler
Programiz PRO
main.py
Visualize
Run
Output
All functional and boundary tests passed.
Closest-pair benchmark:
n=1000: brute evaluations=499500, brute time=0.1348s, divide-and-conquer time=0.0052s
n=4000: brute evaluations=7998000, brute time=2.1758s, divide-and-conquer time=0.0409s
n=8000: brute evaluations=31996000, brute time=8.6187s, divide-and-conquer time=0.0647s
Insertion-sort comparison benchmark:
Nearly sorted comparisons: 1000
Random input comparisons: 251874


Online Python Compiler (Editor
programiz.com/python-programming/online-compiler/
Ask Gemini
Ask Google
School
Programiz Python Online Compiler
Programiz PRO
main.py
Visualize
Run
Output
All functional and boundary tests passed.
Closest-pair benchmark:
n=1000: brute evaluations=499500, brute time=0.1348s, divide-and-conquer time=0.0052s
n=4000: brute evaluations=7998000, brute time=2.1758s, divide-and-conquer time=0.0409s
n=8000: brute evaluations=31996000, brute time=8.6187s, divide-and-conquer time=0.0647s
Insertion-sort comparison benchmark:
Nearly sorted comparisons: 1000
Random input comparisons: 251874





3. Expected Test Result

After running the program, the first line should be:

All functional and boundary tests passed.

The program then prints the benchmark measurements for the three batch sizes and the insertion-sort comparison counts.

The exact elapsed times depend on your computer, so record the actual values shown by your run.

4. GitHub Submission

After testing:

1. Save the file as sentinelops.py.
2. Run it in your Python compiler or terminal.
3. Confirm that all tests pass.
4. Take a screenshot showing the code execution and test results.
5. Upload the file to your GitHub repository.
6. Copy the repository link and submit it through Google Classroom.

Final verification note: Before submitting, test the divide-and-conquer closest-pair implementation against brute force on additional randomly generated small datasets. This is especially important for boundary conditions and equal-distance pairs. Also record the real benchmark outputs from your machine rather than copying illustrative values.